SynthOmnicon: Millennium Barriers

March 28, 2026

0.1 *A Formal Barrier Taxonomy for the Millennium Prize Problems in Lean 4*

Version: v0.1.0 · 2026-03-26 **Authors:** Lando Mills & Large Language Models **Document role:** Self-contained research paper. Presents the machine-checked barrier taxonomy for all seven Clay Millennium Prize Problems, the `BarrierType` inductive, the `ym_is_unique_missing_foundation` theorem, the stacked/parallel sorry distinction, and the primitive bridge connecting sorry boundaries to the SynthOmnicon constraint grammar. Target venue: Journal of Formalized Reasoning / Journal of Automated Reasoning.

The distinction that matters throughout: "we have formalized" is not the same as "we have solved." Every `sorry` at the core of this library is honest. No Millennium Problem is proved here. The contribution is the meta-level structure – what kind of thing each sorry is, and why.

0.2 Three-Document Architecture

The SynthOmnicon Lean library occupies two tracks: `Primitives/` (the 12-primitive constraint grammar) and `Millennium/` (the barrier taxonomy library documented here). This paper reports on the `Millennium/` track and its bridge to `Primitives/`.

Internal references within this paper use §N.

Library location: `SynthOmnicon/Millennium/` – nine files, approximately 1,524 lines. Build target: `lake build Millennium`.

0.3 I. Introduction (v0.1.0, 2026-03-26)

The Clay Mathematics Institute seven Millennium Prize Problems have resisted proof for decades – and in one case, the Riemann Hypothesis, for over a century and a half. Proof assistants have formalized large bodies of mathematics, but existing efforts overwhelmingly target *results that are known*: the Last Theorem of Fermat, the Four Color Theorem, the Kepler Conjecture. Far less attention has been paid to formalizing *why specific problems are hard* – the structural barriers that distinguish them from merely difficult but open problems.

This paper presents a formal barrier taxonomy for all seven Millennium Problems in Lean 4. We make no claim to have solved any of them. The contribution is meta-level: a machine-checked classification of the proof obligations, the structural relationships between them, and their connection to an underlying primitive constraint algebra.

0.3.1 I.1 Why this matters

The distinction between barrier types has practical consequences for the formalization community.

MathlibGap sorries are actionable. A contributor with the right background can in principle discharge them by formalizing a known proof. The `euler_opn_form` sorry in OPN.lean (Euler 1747) and the `mazur_torsion` sorry in BSD.lean (Mazur 1977) are of this type.

OpenProblem sorries define the research frontier. They cannot be discharged without solving the underlying mathematics. Knowing which sorries are of this type – as opposed to merely MathlibGap – is useful for anyone building on the library.

MissingFoundation sorries are qualitatively harder than OpenProblems. An OpenProblem has a well-typed proposition whose truth value is unknown. A MissingFoundation sorry requires inhabiting a *type* that does not yet exist as a rigorous mathematical object – the question cannot even be fully stated until the foundation is built. Yang-Mills is the only Millennium Problem in this category. We prove this formally.

0.3.2 I.2 The sorry depth distinction

We introduce a formal notion of *sorry depth* distinguishing *stacked* from *parallel* proof obligations.

Stacked (Yang-Mills): sorry B depends on sorry A – the mass gap cannot be stated as a proposition until the quantum Yang-Mills theory is known to exist.

Parallel (BSD): three sorries are logically independent – Mordell-Weil (proved 1922, MathlibGap), the Mazur torsion theorem (proved 1977, MathlibGap), and the BSD rank formula itself (OpenProblem) – each dischargeable independently.

Both Yang-Mills and BSD have `sorryDepth = 2`. The structural difference is encoded in the barrier type.

0.3.3 I.3 Contributions

C1 – BarrierType taxonomy: A typed inductive with three constructors, formally distinct (by `decide`), and computably assigned to all seven Millennium Problems.

C2 – ym_is_unique_missing_foundation: A theorem, proved by `decide`, that Yang-Mills is the only Millennium Problem whose barrier is MissingFoundation.

C3 – Stacked vs parallel sorry distinction: Formalized in `sorryDepth` and `ym_has_stacked_not_parallel_`

C4 – NS critical Sobolev exponent: Machine-verified by `norm_num` that $0 < \frac{1}{2} < 1$ – the formal statement of why NS regularity is hard.

C5 – OPN in real Mathlib: the Touchard congruence type-checks using actual `Nat.Perfect` and `ArithmeticFunction.sigma`.

C6 – BSD in real Mathlib: `BSDRankConjecture` stated using actual `WeierstrassCurve` and `IsElliptic`; three parallel sorries formally justified.

C7 – PrimitiveBridge.lean: Formal connection between sorry boundaries and primitive field transitions in the SynthOmnicon grammar; `BarrierPrimitiveCertificate` structure; `primitive_bridge_master` theorem.

0.4 II. Background (v0.1.0, 2026-03-26)

0.4.1 II.1 Lean 4 and Mathlib

All code targets **Lean 4.28.0** with **Mathlib v4.28.0**. We use `import Mathlib.Tactic` throughout plus specific imports per file:

File	Key Mathlib imports
OPN.lean	`Mathlib.NumberTheory.ArithmeticFunction` – `Nat.Perfect`, `ArithmeticFunction.sigma`
BSD.lean	`Mathlib.AlgebraicGeometry.EllipticCurve.Weierstrass`, `.Affine.Point` – `WeierstrassCurve` , `IsElliptic`
YM.lean	`Mathlib.Algebra.Lie.Basic` – `LieRing`, `LieAlgebra`, `LieAlgebra.IsSimple`
NS.lean	`Mathlib.Analysis.InnerProductSpace.Basic` – functional-analytic infrastructure

0.4.2 II.2 The sorry in Lean

In Lean 4, `sorry` is an axiom of type α for any α : Prop (or any type). It allows a proof to typecheck despite containing unfinished obligations. A `sorry`-free proof is fully kernel-verified; a proof with `sorry` is verified modulo those axioms. There is no built-in mechanism in Lean to classify *why* a sorry cannot be discharged. The `BarrierType` inductive in §III provides this classification for the first time.

In the final library, each sorry boundary is declared as an `axiom` rather than left as a literal `sorry`:

```
axiom rh_sorry_boundary : Millennium.RH.RiemannHypothesis
```

This is intentional. An `axiom` is explicit about its foundational status – it extends the logical framework at a named location rather than leaving an opaque hole.

0.4.3 II.3 The SynthOmnicon primitive grammar

The SynthOmnicon framework encodes physical and mathematical systems as 12-tuples $\langle D; T; R; P; F; K; G; \Gamma; \Phi_{\alpha}$ over a constraint algebra. The twelve primitives and their Lean types:

All twelve types derive `DecidableEq`, making structural comparisons computable. The `primitiveMismatches` function (Hamming distance over twelve fields) is proved bounded by 12 and zero iff tuples are identical – all by `decide` or `omega`.

0.5 III. The Barrier Taxonomy (v0.1.0, 2026-03-26)

0.5.1 III.1 The BarrierType inductive

```
inductive BarrierType
  | MathlibGap
  | OpenProblem
  | MissingFoundation
  deriving Repr, DecidableEq
```

The three constructors represent qualitatively distinct epistemic states.

`MathlibGap`: The theorem has been proved in the mathematical literature. A Lean proof exists *in principle* using currently available tools; the sorry persists only because no one has yet carried out the formalization. Such sorries are *defeasible* – they will eventually be discharged as Mathlib grows. Examples: the Euler 1747 characterization of OPN structure (`euler_opn_form`); the Mazur 1977 torsion theorem (`mazur_torsion`).

Primitive	Lean type	Role
D **(Dimensionality)**	`Dimensionality`	Spatial encoding: point / line / cube / holographic
T **(Topology)**	`Topology`	Connectivity: linear / branched / network / holographic
R **(Recognition)**	`Recognition`	Symmetry / matching mode
P **(Polarity)**	`Polarity`	Charge / orientation
Γ **(Grammar)**	`Grammar`	Composition rule
F **(Fidelity)**	`Fidelity`	Information reliability: noise / ℓ / η / $\hbar$
K **(Kinetic character)**	`KineticChar`	Rate / trapping: fast / mod / slow / trap / MBL
G **(Granularity)**	`Granularity`	Description scale: $\aleph$ / $\beth$ / $\gimel$
Φ_c **(Criticality)**	`Criticality`	Phase position: sub / critical / super
H **(Chirality)**	`Chirality`	Temporal asymmetry depth: H_0 / H_1 / H_2 / H_∞
S **(Stoichiometry)**	`Stoichiometry`	Ratio: 1:1 / 1:n / n:m / catalytic
Ω **(Protection)**	`Protection`	Topological protection: 0 / Z_2 / Z / C / NA

`OpenProblem`: No proof of the theorem exists anywhere. The sorry cannot be discharged because the underlying mathematics is unsolved. Examples: the Riemann Hypothesis, the BSD rank conjecture, the nonexistence of odd perfect numbers.

`MissingFoundation`: The sorry requires inhabiting a *type* that does not yet exist as a rigorous mathematical object. The distinction from `OpenProblem` is ontological: an `OpenProblem` has a well-typed proposition – we know what it means, we just do not know if it is true. A `MissingFoundation` sorry cannot even be fully stated until the missing type is constructed. Example: `PathIntegralMeasure G` in Yang-Mills – the path integral measure over gauge connections modulo gauge equivalence in four Euclidean dimensions.

Formal distinctness is immediate:

```
theorem missing_foundation_vs_open_problem :
    BarrierType.MissingFoundation != BarrierType.OpenProblem := by
   decide
```

0.5.2 III.2 The master classification

```
def millenniumBarrier : MillenniumProblem -> BarrierType
  | .RH    => .OpenProblem
  | .Hodge => .OpenProblem
  | .PvsNP => .OpenProblem
  | .NS    => .OpenProblem
  | .YM    => .MissingFoundation
  | .BSD   => .OpenProblem
  | .OPN   => .OpenProblem
```

The central theorem of the library:

```
theorem ym_is_unique_missing_foundation :
    forall p : MillenniumProblem,
      millenniumBarrier p = .MissingFoundation -> p = .YM := by
  intro p hp; cases p <;> simp_all [millenniumBarrier]
```

Yang-Mills is the only Millennium Problem with a `MissingFoundation` barrier. This is a theorem about the structure of mathematics, proved by exhaustive case analysis over the seven-constructor `MillenniumProblem` inductive, closed by `simp_all`.

0.5.3 III.3 Sorry depth

```
def sorryDepth : MillenniumProblem ->
  | .RH    => 1   -- ZeroFreeStrip 0 (one sorry; everything else
   proved)
  | .Hodge => 1   -- algebraic cycle representative
  | .PvsNP => 1   -- circuit lower bound
  | .NS    => 1   -- global regularity certificate
  | .YM    => 2   -- existence (sorry 1) THEN mass gap (sorry 2)
  | .BSD   => 2   -- three parallel; Mordell-Weil + Mazur torsion +
   BSD formula
  | .OPN   => 2   -- Euler form (MathlibGap) + nonexistence (
   OpenProblem)
```

The key structural distinction:

```
theorem ym_has_stacked_not_parallel_sorries :
    sorryDepth .YM = sorryDepth .BSD and
    millenniumBarrier .YM = .MissingFoundation and
    millenniumBarrier .BSD = .OpenProblem := by
  simp [sorryDepth, millenniumBarrier]
```

YM and BSD share `sorryDepth = 2` but for different structural reasons: in YM, sorry 2 (mass gap) logically depends on sorry 1 (existence of the theory). In BSD, the three sorries are logically independent – each can in principle be discharged separately.

0.6 IV. Per-Problem Analysis (v0.1.0, 2026-03-26)

0.6.1 IV.1 Riemann Hypothesis

Barrier: `OpenProblem` · Missing type: `ZeroFreeStrip 0`

The file introduces abstract types `RiemannZeta`, `CriticalStrip`, and `ZeroFreeStrip` to provide typed scaffolding. The sorry boundary is the statement that $\zeta(s) \neq 0$ for all s with $0 < \mathrm{Re}(s) < 1$, $\mathrm{Re}(s) \neq \frac{1}{2}$.

```
axiom rh_sorry_boundary : Millennium.RH.RiemannHypothesis

theorem rh_barrier : RiemannHypothesis <-> ZeroFreeStrip 0
```

The barrier theorem isolates `ZeroFreeStrip 0` as the exact missing type: the sorry is tight – RH is equivalent to inhabiting that type and nothing else.

0.6.2 IV.2 Yang-Mills Existence and Mass Gap

Barrier: `MissingFoundation` · *Missing type (primary):* `PathIntegralMeasure G`

The YM file is distinguished by *two stacked* sorries. The axiom bundles both:

```
axiom ym_sorry_boundary :
    forall (G : Type*) [LieRing G] [LieAlgebra  G] [LieAlgebra.
   IsSimple  G],
      Nonempty (QuantumYMTheory G) and
      forall T : QuantumYMTheory G, 0 < massGap G T
```

`ym_theory_exists` (MissingFoundation) is annotated with sorry first. `ym_mass_gap` (OpenProblem) depends on the first – it is statable only once the theory exists. The `PathIntegralMeasure G` type is defined to require `LieAlgebra.IsSimple`, reflecting that the problem is well-posed only for compact simple gauge groups.

Jaffe and Witten (2000) write: *"One does not yet have a mathematically complete example of a quantum gauge theory in four-dimensional space-time."* The `MissingFoundation` constructor formalizes exactly this observation.

0.6.3 IV.3 Hodge Conjecture

Barrier: `OpenProblem` · *Missing type:* `AlgebraicCycleRep X p alpha`

The file introduces `SmoothProjectiveVariety`, `HodgeCohomology X p`, and `AlgebraicCycle X p`. The missing type is cycle class surjectivity: a proof that $\alpha \in H^{2p}(X, \mathbb{Q}) \cap H^{p,p}$ lies in the image of $\mathrm{cl} : \mathrm{CH}^p(X) \otimes \mathbb{Q} \to H^{2p}(X, \mathbb{Q})$.

The layer structure is explicit:

Layer 1 (MathlibGap): The $p = 1$ case (Lefschetz 1924) is documented as dischargeable – the theorem is proved, merely not yet in Mathlib.

Layer 2 (OpenProblem): The $p \geq 2$ case, which is the actual Hodge conjecture.

```
theorem hodge_barrier :
    HodgeConjecture <-> forall (X : SmoothProjectiveVariety) (p : ),
      forall alpha : HodgeCohomology X p, AlgebraicCycleRep X p
   alpha
```

0.6.4 IV.4 Navier-Stokes

Barrier: `OpenProblem` *(near-*`MissingFoundation`*)* · *Missing type:* `GlobalRegularityCert u`

The central formal result is a machine-verified statement of why NS is hard:

```
def CriticalSobolevExponent :  := 1 / 2

theorem energy_norm_subcritical    : 0 < CriticalSobolevExponent
   := by norm_num
theorem enstrophy_norm_supercritical : CriticalSobolevExponent < 1
   := by norm_num

theorem critical_scaling_gap :
    0 < CriticalSobolevExponent and CriticalSobolevExponent < 1 :=
  \langleenergy_norm_subcritical, enstrophy_norm_supercritical\
   rangle
```

Interpretation: The energy norm ($H^0 = L^2$, exponent $s = 0$) is subcritical under the natural scaling of the 3D NS equations; the enstrophy norm (H^1, exponent $s = 1$) is supercritical. The critical Sobolev exponent $s = \frac{1}{2}$ sits strictly between them. Global regularity requires controlling the $\dot{H}^{1/2}$ norm – which is exactly what is not known. Two `norm_num` calls verify this formally.

Three tiers of sorry are distinguished: `leray_weak_existence` (MathlibGap, Leray 1934); `ns_small_data_global_regularity` (MathlibGap, Koch-Tataru 2001); `ns_certificate` (OpenProblem, the Clay problem itself).

0.6.5 IV.5 P versus NP

Barrier: `OpenProblem` (with deep `MathlibGap` in complexity formalization) · Missing type: `CircuitLowerBound` ϵ

The file uses concrete types to ground the statement without relying on complexity classes that Mathlib does not yet have:

```
def BooleanCircuit (n : ) := (Fin n -> Bool) -> Bool
def CNFFormula (n : )     := List (List (Literal n))
```

Three **meta-barriers** are formalized as `trivial` theorems with full documentation:

```
-- Baker-Gill-Solovay (1975): P vs NP is independent of relativized
   computation.
theorem bgs_relativization_barrier : True := trivial

-- Razborov-Rudich (1994): ''Natural proofs'' cannot separate P from
    NP
-- unless one-way functions do not exist.
theorem razborov_rudich_natural_proofs_barrier : True := trivial

-- Aaronson-Wigderson (2009): Algebrizing techniques cannot separate
    P from NP.
theorem aaronson_wigderson_algebrization_barrier : True := trivial
```

The `trivial` bodies are honest: these are theorems *about* proof techniques, not the conjecture itself. They are documented constraints on the proof search space.

0.6.6 IV.6 Odd Perfect Number

Barrier: `OpenProblem` (primary); `MathlibGap` (Euler form layer) · Missing type: `OPNConjecture`

This file uses the **actual Mathlib infrastructure**:

```
import Mathlib.NumberTheory.ArithmeticFunction
-- Nat.Perfect :  -> Prop
-- Nat.ArithmeticFunction.sigma : ArithmeticFunction
```

The multiplicativity of σ is proved using the actual Mathlib lemma:

```
theorem sigma_multiplicative (a b : ) (h : Nat.Coprime a b) :
    (Nat.ArithmeticFunction.sigma 1) (a * b) =
    (Nat.ArithmeticFunction.sigma 1) a *
    (Nat.ArithmeticFunction.sigma 1) b :=
  Nat.ArithmeticFunction.IsMultiplicative.sigma.map_mul_of_coprime h
```

The two-layer structure: `euler_opn_structure` (Euler 1747, MathlibGap – every OPN has the form $N = p^\alpha m^2$ with $p \equiv \alpha \equiv 1 \pmod 4$) stacked methodologically (not logically) with `opn_nonexistence` (OpenProblem). The current lower bound $N > 10^{1500}$ (Ochem-Rao 2012) is documented as a MathlibGap sorry. The companion file `OPN_2adic.lean` in `Primitives/` contains the full machine-verified Touchard congruence.

0.6.7 IV.7 Birch and Swinnerton-Dyer

Barrier: `OpenProblem` (primary); `MathlibGap` (two layers) · Missing type: `BSDRankCertificate W`

BSD.lean is the most concretely grounded file, using real Mathlib types throughout:

```
import Mathlib.AlgebraicGeometry.EllipticCurve.Weierstrass
import Mathlib.AlgebraicGeometry.EllipticCurve.Affine.Point

def ExampleCurve : WeierstrassCurve  :=
  { a := 0, a := 0, a := 0, a := -1, a := 0 }
-- y^2 = x§ - x, the congruent number curve for n = 1

def BSDRankConjecture : Prop :=
    forall (W : WeierstrassCurve ) [W.IsElliptic],
      ellipticRank W = analyticRank W
```

The three sorries are formally **parallel** – logically independent:

Sorry	Type	Barrier	Attribution
`mordell_weil`	`MordellWeilGroup`	MathlibGap	Mordell 1922: $E(\mathbb{Q})$ finitely generated
`mazur_torsion`	`MazurTorsionBound`	MathlibGap	Mazur 1977: torsion subgroup classification
`bsd_certificate`	`BSDRankCertificate`	OpenProblem	The BSD conjecture itself

Known partial results are documented: Coates-Wiles 1977 (MathlibGap, rank 0 for CM curves); Gross-Zagier + Kolyvagin (MathlibGap, BSD holds for analytic rank ≤ 1).

0.7 V. The Primitive Bridge (v0.1.0, 2026-03-26)

[Connects Millennium/ and Primitives/. Full Lean source: `SynthOmnicon/Millennium/PrimitiveBridge.lean`

0.7.1 V.1 Motivation

The barrier taxonomy classifies *why* sorries cannot be discharged but does not explain *what structural feature* of each problem generates its barrier. `PrimitiveBridge.lean` provides this second layer: a machine-checked connection between each sorry boundary and a specific primitive field transition in the SynthOmnicon grammar.

The key claim: every Millennium Problem sorry boundary corresponds to a specific field value that the natural primitive encoding of the problem *wants* but cannot obtain given its surrounding constraints. This is testable – the encodings are concrete `Synthon` values, field values are decidable, and barrier classifications are machine-checked.

0.7.2 V.2 BarrierPrimitiveCertificate

```
structure BarrierPrimitiveCertificate (p : Barriers.
   MillenniumProblem) where
  encoding      : Synthon
  blockedField  : String
  barrier       : Barriers.BarrierType
  barrier_correct : barrier = Barriers.millenniumBarrier p
```

The `barrier_correct` field is a proof obligation, not documentation. For each concrete problem `p`, `millenniumBarrier p` reduces definitionally, making `barrier_correct := rfl` a machine-checked verification. Three instances:

```
def ym_certificate : BarrierPrimitiveCertificate .YM where
  encoding     := ym_quantum_target
  blockedField := ''gran: G_beth -> G_aleph (PathIntegralMeasure G
   missing)''
  barrier      := .MissingFoundation
  barrier_correct := rfl   -- Lean verifies: .MissingFoundation =
   millenniumBarrier .YM (ok)

def opn_certificate : BarrierPrimitiveCertificate .OPN where
  ...
  barrier_correct := rfl   -- .OpenProblem = millenniumBarrier .OPN
   (ok)

def ns_certificate : BarrierPrimitiveCertificate .NS where
  ...
  barrier_correct := rfl   -- .OpenProblem = millenniumBarrier .NS (
   ok)
```

0.7.3 V.3 The YM primitive barrier

The Yang-Mills quantum lift is encoded as the transition from `ym_classical` (four-dimensional gauge theory before quantization) to `ym_quantum_target` (what YM would look like if the path integral existed). Four primitive fields change:

Field	Classical YM	Quantum target	Interpretation
F **(Fidelity)**	F_η	$F_\hbar$	Classical → quantum coherence
K **(Kinetic)**	K_{mod}	K_{trap}	Perturbative → confinement
G **(Granularity)**	$G_\beth$	$G_\aleph$	Mesoscale local → quantum fine-grained
Φ_c **(Criticality)**	Φ_{sub}	Φ_c	No gap → mass gap (critical phenomenon)

```
theorem ym_classical_to_quantum_cost :
    primitiveMismatches ym_classical ym_quantum_target = 4 := by
   decide
```

The $G_\beth \to G_\aleph$ **transition** is the primitive certificate of the missing `PathIntegralMeasure`: · $G_\beth$ = mesoscale local description – classical gauge field theory describable without integration

over all gauge connections. · $G_\aleph$ = quantum fine-grained – requires a well-defined measure on the full gauge connection space modulo gauge equivalence. · Constructing `PathIntegralMeasure G` *is* providing the $G_\aleph$ description.

Crucially, the quantum target stays at `D_cube` (four-dimensional, local) rather than `D_holo` (holographic):

```
theorem ym_quantum_target_is_local : ym_quantum_target.dim = D_cube
   := rfl
theorem ym_qg_dim_differ : ym_quantum_target.dim != quantum_gravity.
   dim := by decide
```

This formally distinguishes the YM problem from quantum gravity. Quantizing YM does not require holographic substrate – the two problems are structurally distinct synthons, with the YM quantum target at Hamming distance ≥ 1 from `quantum_gravity`.

The full certificate:

```
theorem ym_primitive_barrier_certificate :
    ym_quantum_target.gran = G_aleph and     -- needs quantum-level
   granularity
    ym_quantum_target.crit = Phi_c and       -- needs mass gap (
   critical)
    ym_quantum_target.fid  = F_hbar and      -- needs quantum
   fidelity
    ym_quantum_target.kin  = K_trap and      -- needs confinement
    ym_quantum_target.dim  = D_cube and      -- stays 4D local (NOT
   QG)
    ym_quantum_target.dim  != quantum_gravity.dim and
    Barriers.millenniumBarrier .YM = .MissingFoundation := by
  exact \langlerfl, rfl, rfl, rfl, rfl, by decide, rfl\rangle
```

0.7.4 V.4 Other primitive certificates

The OPN, NS, and RH certificates connect to the Φ_c criticality primitive:

Problem	Encoding field	Value	Structural meaning
OPN	`crit` + `kin`	$\Phi_c + K_{\text{trap}}$	$\sigma(n) = 2n$ is exact criticality; overdetermined constraint system has no relaxation path
NS	`crit`	Φ_{sub}	Smooth solutions stay subcritical; the sorry proves they never cross to Φ_c (blow-up threshold)
RH	`crit`	Φ_c	The critical line $\text{Re}(s) = \frac{1}{2}$ is the Φ_c locus of the zeta function

```
theorem opn_primitive_certificate :
    opn_encoding.crit = Phi_c and opn_encoding.kin = K_trap and
```

```
    Barriers.millenniumBarrier .OPN = .OpenProblem := \langlerfl,
   rfl, rfl\rangle

theorem ns_primitive_certificate :
    ns_encoding.crit = Phi_sub and
    Barriers.millenniumBarrier .NS = .OpenProblem := \langlerfl, rfl
   \rangle

theorem rh_primitive_certificate :
    rh_encoding.crit = Phi_c and
    Barriers.millenniumBarrier .RH = .OpenProblem := \langlerfl, rfl
   \rangle
```

0.7.5 V.5 The master bridge theorem

```
theorem primitive_bridge_master :
    primitiveMismatches ym_classical ym_quantum_target = 4 and
    Barriers.millenniumBarrier .YM = .MissingFoundation and
    opn_encoding.crit = Phi_c and opn_encoding.kin = K_trap and
    Barriers.millenniumBarrier .OPN = .OpenProblem and
    ns_encoding.crit = Phi_sub and
    Barriers.millenniumBarrier .NS = .OpenProblem and
    rh_encoding.crit = Phi_c and
    Barriers.millenniumBarrier .RH = .OpenProblem :=
  \langleby decide, rfl, rfl, rfl, rfl, rfl, rfl, rfl, rfl\rangle
```

This theorem machine-checks four barrier certificates simultaneously. The `by decide` term computes the Hamming distance between two concrete `Synthon` values over twelve decidable fields. All `rfl` terms verify that `millenniumBarrier` reduces correctly on each concrete constructor.

0.8 VI. Proof Engineering Notes (v0.1.0, 2026-03-26)

0.8.1 VI.1 decide vs norm_num vs rfl

Three automation tactics drive the machine-checked results:

`rfl` – Used for definitional equalities. `millenniumBarrier .YM = .MissingFoundation` reduces by `rfl` because `millenniumBarrier` is a `def` (not `opaque`) that pattern-matches on `MillenniumProblem`. All `barrier_correct := rfl` in `BarrierPrimitiveCertificate` instances are of this type.

`decide` – Used for decidable propositions over finite types. `primitiveMismatches ym_classical ym_quantum_target = 4` is proved by kernel reduction of the computable `primitiveMismatches` function over two concrete `Synthon` values. `ym_is_unique_missing_foundation` is closed by `cases p <;> simp_all`.

`norm_num` – Used for real arithmetic in NS.lean: `0 < (1:)/2` and `(1:)/2 < 1`. These are the only results requiring real-number reasoning; all other automation stays in the decidable fragment.

0.8.2 VI.2 Abstract vs concrete type design

A recurring design choice is how much to use abstract axiomatized types versus concrete Mathlib types.

BSD uses the most concrete types: `WeierstrassCurve` , `IsElliptic`. This grounds the

conjecture in actual Mathlib infrastructure. The tradeoff: `MordellWeilGroup`, `ellipticRank`, and `analyticRank` must be axiomatized, but they are axiomatized *over* a real Mathlib type, not a fictional one.

YM uses the most abstract types: `PathIntegralMeasure`, `QuantumYMTheory`, `massGap` are all axiomatized from scratch. This is unavoidable – there is no Lean type for a rigorous path integral measure in 4D, which is precisely the `MissingFoundation` claim.

OPN uses the most Mathlib: `Nat.Perfect`, `ArithmeticFunction.sigma`, `IsMultiplicative.sigma.map_m` The `sigma_multiplicative` theorem is a thin wrapper over existing machinery.

0.8.3 VI.3 The sorry-as-axiom encoding

Each sorry boundary is declared as an `axiom` rather than a `sorry` in the final library. An `axiom` in Lean explicitly extends the logical framework at a named, typed location. The library does not contain accidental sorries from incomplete proof attempts – every sorry-equivalent obligation is a named, typed axiom at a precise location, documented with its barrier classification.

0.9 VII. Related Work (v0.1.0, 2026-03-26)

0.9.1 VII.1 Formalization of known results

The ongoing Lean formalization of the Last Theorem of Fermat (2024-ongoing) provides a template for long-term deep formalization. Our work is complementary: we formalize the barrier structure of *open* problems rather than the proof of a closed one.

Several groups have worked on BSD-adjacent results in Lean and Coq. Partial formalizations of Mordell-Weil exist but none are complete in Mathlib as of v4.28.0. BSD.lean is, to our knowledge, the first statement of the BSD conjecture using the actual Mathlib `WeierstrassCurve` infrastructure.

For Navier-Stokes, the PFR (Polynomial Freiman-Ruzsa) formalization demonstrated that deep analysis results can be machine-verified in Lean. Our NS.lean targets specifically the critical scaling structure rather than a specific regularity result.

0.9.2 VII.2 Proof complexity and barrier theory

The mathematical theory of proof complexity barriers – relativization (Baker-Gill-Solovay 1975), natural proofs (Razborov-Rudich 1994), algebrization (Aaronson-Wigderson 2009) – is extensive. We are not aware of any prior formalization of these barriers in a proof assistant. PvsNP.lean formalizes them as `True := trivial` with full documentation: an honest encoding of theorems *about* proof search, not the conjecture itself.

0.9.3 VII.3 Prior sorry taxonomies

We are not aware of any prior work that formalizes a taxonomy of *why* sorries cannot be discharged. The distinction between MathlibGap, OpenProblem, and MissingFoundation appears to be novel as a typed Lean inductive. The closest conceptual relative is Jaffe-Witten observation that the YM path integral measure "does not exist as a mathematical object." We formalize this intuition in the `MissingFoundation` constructor and prove it uniquely applies to YM among the seven Millennium Problems.

0.9.4 VII.4 The SynthOmnicon framework

The SynthOmnicon 12-primitive grammar is developed in companion grammar documents (SynthOmnicon repository). The `Primitives/` track of the Lean library – `Core.lean`, `Synthon.lean`, `TierCrossing.lean`, `OPN_2adic.lean`, `BSD_2adic.lean` – constitutes a parallel formalization

effort. The present paper uses `Core.lean` and `Synthon.lean` only for `PrimitiveBridge.lean`; the full primitive algebra is not required for the barrier taxonomy itself.

0.10 VIII. Discussion (v0.1.0, 2026-03-26)

0.10.1 VIII.1 The MissingFoundation barrier and mathematics

The `MissingFoundation` barrier for Yang-Mills reflects something philosophically significant: the quantum Yang-Mills problem is not just an unsolved theorem but an unsolved *definition*. Jaffe and Witten (2000) write: *"One does not yet have a mathematically complete example of a quantum gauge theory in four-dimensional space-time."*

This is qualitatively different from the Riemann Hypothesis, where the statement $\zeta(s) \neq 0$ for $0 < \mathrm{Re}(s) < 1$, $\mathrm{Re}(s) \neq \frac{1}{2}$ is perfectly well-typed – we just do not know its truth value. For Yang-Mills, the sentence "the quantum YM theory exists and has mass gap $\Delta > 0$" cannot be made rigorous until the path integral measure is constructed.

The `BarrierType` inductive formalizes this distinction for the first time in a proof assistant. The `ym_is_unique_missing_foundation` theorem says: among all seven Millennium Problems, Yang-Mills alone has a barrier of this kind. Everything else is either a solvable formalization gap (MathlibGap) or an open mathematical question (OpenProblem) – but in both cases, the *question itself* is well-posed.

0.10.2 VIII.2 What the primitive bridge adds

The `PrimitiveBridge.lean` connection adds a second level of explanation: *why* does the missing type correspond to a specific missing primitive? The claim is not that the primitive grammar *causes* the open problem – it is that the grammar *encodes* the structural reason precisely. The $G_{\beth} \to G_{\aleph}$ transition for YM is a formal statement that the classical mesoscale-local description is insufficient and a quantum-level fine-grained description is required. This is exactly what `PathIntegralMeasure` provides.

The machine-checked nature of `primitive_bridge_master` means this is not merely commentary. Four barrier certificates are verified simultaneously, and the quantum YM lift cost of 4 primitive mismatches is a theorem, not an estimate.

0.10.3 VIII.3 Limitations

Every sorry-boundary in the library is genuine. The Lean files typecheck because the barriers are declared as axioms; removing them would leave genuine proof obligations that cannot currently be discharged.

The primitive encodings in `PrimitiveBridge.lean` are our best formalization of the structural content of each problem – but they are choices. Another author might make different encoding decisions for Hodge or P vs NP. The `BarrierType` taxonomy and `ym_is_unique_missing_foundation` are more robust: they depend only on the `MillenniumProblem` inductive and `millenniumBarrier`, not on specific encodings.

0.11 IX. Future Work (v0.1.0, 2026-03-26)

0.11.1 IX.1 Discharge of MathlibGap sorries

The most actionable near-term work: `euler_opn_form` (OPN.lean) – all required tools are in `OPN_2adic.lean` and the proof is feasible with current Mathlib. `mazur_torsion` (BSD.lean) and `mordell_weil` (BSD.lean) are longer-term Mathlib development targets. `lefschetz_11` (Hodge.lean) requires complex geometry formalization.

0.11.2 IX.2 Lattice instances for Criticality

The `Criticality` primitive has an unusual meet semantics: Φ_c is absorbing under meet ($\Phi_c \sqcap x = \Phi_c$ for all x), which contradicts the `min`-derived order. A correct `MeetSemilattice` instance requires overriding the default. Once in place, the OPN and NS bridge encodings can be connected more tightly to the primitive algebra.

0.11.3 IX.3 Extending the bridge to all seven problems

`PrimitiveBridge.lean` provides certificates for YM, OPN, NS, and RH. Hodge (R-degeneracy as topology-to-algebra lift), P vs NP (K_{trap} blocking low-complexity), and BSD (Φ_c as rank charge-carrier) require either more complex primitive signatures or narrative justification that is harder to encode compactly. Extending the bridge to all seven is a natural next step.

0.11.4 IX.4 Formalizing the meta-barriers

The three `trivial` theorems in PvsNP.lean (BGS relativization, Razborov-Rudich natural proofs, Aaronson-Wigderson algebrization) should eventually be given genuine Lean statements. This requires formalizing oracle Turing machines, circuit complexity classes, and algebraic extension closure – a substantial independent Mathlib development effort.

0.12 X. Conclusion (v0.1.0, 2026-03-26)

We have presented a nine-file Lean 4 library formalizing a barrier taxonomy for the seven Clay Millennium Prize Problems. The central contribution is the `BarrierType` inductive (MathlibGap / OpenProblem / MissingFoundation) and the machine-checked theorem that Yang-Mills is the unique Millennium Problem with a `MissingFoundation` barrier. We have formally distinguished stacked from parallel sorry depth, machine-verified the critical Sobolev scaling in Navier-Stokes by `norm_num`, grounded BSD in the actual Mathlib `WeierstrassCurve` infrastructure, grounded OPN in the actual Mathlib `Nat.Perfect` and `ArithmeticFunction.sigma`, and connected sorry boundaries to primitive field transitions via a machine-checked bridge file.

The `sorry` in a Lean proof is usually treated as an obstacle to be removed. This library treats it as a datum to be classified. The three barrier types represent three distinct relationships between human mathematics and the proof assistant: what we can formalize but have not, what we have not solved, and what we have not yet defined. Formally distinguishing these is the first step toward a systematic theory of the frontier of formalized mathematics.

0.13 Appendix I. Library Structure

```
SynthOmnicon/Millennium/
  RH.lean              ~110 lines   OpenProblem . ZeroFreeStrip
  YM.lean              ~140 lines   MissingFoundation .
   PathIntegralMeasure (stacked)
  Hodge.lean           ~130 lines   OpenProblem . AlgebraicCycleRep
  NS.lean              ~160 lines   OpenProblem .
   GlobalRegularityCert . norm_num
  PvsNP.lean           ~170 lines   OpenProblem . CircuitLowerBound
   . meta-barriers
  OPN.lean             ~150 lines   OpenProblem . Nat.Perfect .
   sigma_multiplicative
  BSD.lean             ~200 lines   OpenProblem . WeierstrassCurve
   (parallel)
  Barriers.lean        ~224 lines   Taxonomy . ym_is_unique_missing_
   foundation
```

```
  PrimitiveBridge.lean ~230 lines   Bridge .
   BarrierPrimitiveCertificate . master theorem

SynthOmnicon/Primitives/  (companion track, used by PrimitiveBridge
   only)
  Core.lean            ~213 lines   12 primitive inductive types .
   DecidableEq
  Synthon.lean         ~259 lines   Synthon struct .
   primitiveMismatches . P-70 . SM/QG
```

Total (Millennium track): approximately 1,524 lines. Build: `lake build Millennium`.

0.14 Appendix II. Key Theorems

Theorem	File	Proof	Significance
`ym_is_unique_missing_`	Barriers	`cases` + `simp_all`	YM uniquely lacks a foundational object
`missing_foundation_vs`	Barriers	`decide`	Barrier types are formally distinct
`ym_has_stacked_not_pa`	Barriers	`simp`	Stacked $\neq$ parallel at same depth
`critical_scaling_gap`	NS	`norm_num`	$0 < \frac{1}{2} < 1$ in $\mathbb{R}$
`sigma_multiplicative`	OPN	Mathlib API	$\sigma(ab) = \sigma(a)\sigma(b)$ for coprime
`ym_classical_to_quant = 4`	PrimitiveBridge	`decide`	Quantum lift costs 4 primitive changes
`ym_primitive_barrier_`	PrimitiveBridge	`rfl` + `decide`	$G_{\beth} \to G_{\aleph}$ = PathIntegralMeasure
`primitive_bridge_mast`	PrimitiveBridge	`decide` + `rfl`	Four certificates simultaneously
`ym_opn_barrier_distin`	PrimitiveBridge	`decide`	MissingFoundation $\neq$ OpenProblem at problem level
`sm_qg_distance_exact = 9`	Synthon	`decide`	SM/QG Hamming distance machine-verified

End of MILLENNIUM_BARRIERS_PAPER.md v0.1.0

Draft 2026-03-26 · Lean 4.28.0 · Mathlib v4.28.0 · Target: Journal of Formalized Reasoning / JAR